

G1 Unitree — Autonomous Navigation & Meta-Reasoning (consumer “Air”, no EDU/DDS)

Autonomous exploration and obstacle avoidance on a **stock Unitree G1 “Air”** humanoid — a consumer unit that exposes **only a single WebRTC session through the official app** (no ROS 2, no DDS, no SSH, no low-level SDK). Everything here is built **on top of that one channel**, by tapping the app’s own WebView over USB.

The robot maps and navigates while we read its LiDAR/odometry and drive it — all without EDU hardware — and learns from its own collisions. The longer-term goal is **meta-reasoning**: a robot that reasons about the reliability of its own perception and adapts.

TL;DR — what this does

- **Reads the robot’s SLAM live** (point cloud + odometry) from Python, over USB, by hooking the app’s WebView with `ios-webkit-debug-proxy` (no second WebRTC session needed).
- **Drives the robot** (walk / turn / strafe) by **injecting `rt/wirelesscontroller` velocity commands into the app’s existing WebRTC datachannel** — the only way to move it programmatically without a 2nd session.
- **Autonomous explore mode**: wanders a room, builds coverage, avoids obstacles fusing **LiDAR + camera** (floor segmentation, edge, YOLOv8s, MiDaS depth) + **odometry collision detection**, and **remembers obstacles it bumps into**.
- **Live visualizer** of the LiDAR cloud (2D/3D) + robot pose + path + camera with YOLO boxes.

This was reverse-engineered against the owner’s **own** robot/app for interoperability. The APK, keys and proprietary assets are **not** redistributed here.

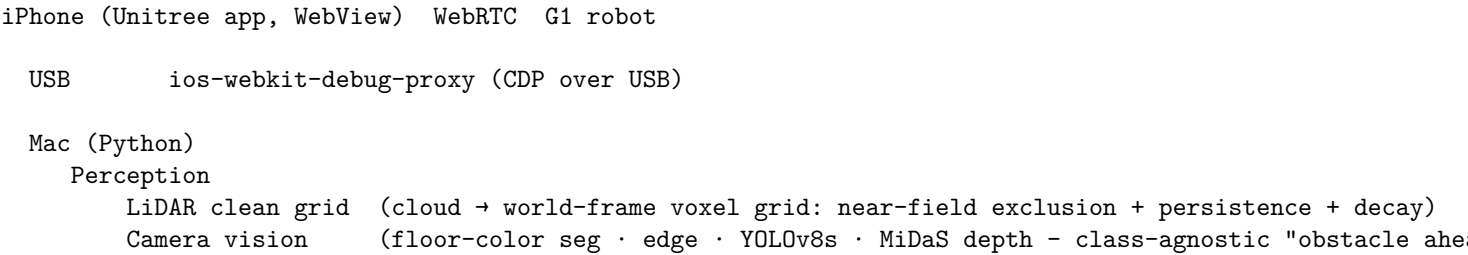
Why it’s hard (the constraints)

The G1 Air is **not** the EDU/developer unit:

Want	Air reality
ROS 2 / DDS / CycloneDDS SSH / low-level SDK (<code>rt/arm_sdk</code> , <code>lowcmd</code>)	internal bus not exposed on the WiFi AP (EDU-only) forwarded but not actuated
Raw LiDAR cloud over WebRTC	only <code>odom</code> + <code>slam_info</code> reach a 3rd-party client; the dense cloud is decoded only inside the app’s WebView
Two WebRTC sessions (one app, one ours)	robot allows one peer — the app holds it

So the trick is: **don’t fight the app — live inside it**. We attach to the WebView’s JS over USB (immune to the robot AP’s client isolation), read the decoded cloud + odom, and publish velocity on the app’s own datachannel.

Architecture



```
Collision sensor (odometry stall = bumped something no sensor saw)
Control (inject rt/wirelesscontroller {lx,ly,rx,ry} @20Hz, in-page driver + dead-man)
Exploration (reactive wander + coverage novelty + redirection to unexplored)
```

Coordinate note: the cloud is in **Three.js Y-up** frame (the app's renderer), the odometry is in **ROS Z-up** frame — they are reconciled in code (`cloud_x odom_x, height=idx1, cloud_z=-odom_y`).

The scripts

Navigation / autonomy - g1_nav.py — the main program. Unified capture + control + exploration. Modes: - **watch** — live odom + cloud point count (read-only) - **clr** — read-only obstacle clearances around the robot (LiDAR grid) - **vsee** — read-only camera vision readout (floor fractions, YOLO label, MiDaS depth_ratio) - **forward N / turn DEG / gore1 F L / goto X Y** — closed-loop primitives (odom feedback) - **nav X Y / navrel F L** — go to a point **with reactive obstacle avoidance** - **explore [secs]** — **autonomous mapping/exploration** (the headline feature) - **g1_inject_teleop.py** — option-C teleop injection (sniff/capture/drive) — proof that we can move the robot via the app's datachannel - **g1_teleop.py** — direct walking via `rt/wirelesscontroller` (app closed)

Perception / viz - g1_inspector_bridge.py — live LiDAR cloud (2D/3D) + pose + path + **camera window with YOLO boxes** - **g1_cam_probe.py** — probe the app's `<video>` element (camera) - **slam_g1_mapping.py** — start/stop/save SLAM from Python (app closed) - **g1_slam_viz.py, g1_map_viz.py** — odometry / saved-map visualizers

Reverse-engineering / diagnostics - dump_services.py, discover_slam_api.py, query_api.py, g1_slaminfo_dump.py, ... — how the API was mapped

Docs - AUTONOMOUS_NAVIGATION.md — perception/control/exploration **algorithm** + a roadmap of improvements (incl. the meta-reasoning direction) - **G1_Air_SLAM_SOLVED.md / .pdf** — the SLAM/WebRTC reverse-engineering writeup

Quick start

Prereqs (on the Mac, in a venv):

```
brew install ios-webkit-debug-proxy
pip install websocket-client requests numpy matplotlib pillow ultralytics timm
# (MiDaS depth pulls its weights on first run; runs on Apple-GPU/MPS automatically)
```

Bring-up: 1. iPhone: Unitree app connected to the robot, **standing**, on the **SLAM/map screen, camera on**. iPhone Web Inspector ON; **don't** open Safari's inspector on that page (one debugger per page). 2. USB-connect the iPhone to the Mac (trust). 3. Terminal 1: `ios_webkit_debug_proxy` 4. Terminal 2: `bash python g1_nav.py watch # confirm odometry is live (x/y/yaw change when you move the robot) python g1_nav.py explore 90 # autonomous exploration`

Safety: keep the physical remote in hand as a kill switch ($L2+B$ = damping/stop), clear 2–3 m of space, and start with the robot freshly charged (>80%) and standing in walk mode.

Learning from failure (toward meta-reasoning)

Each real collision is treated as a perception signal: - **Memory:** the bumped obstacle (which the LiDAR couldn't see) is *injected into the obstacle grid* so the robot doesn't hit it again. - **Dataset:** every collision saves a camera snapshot + a `.txt` of *what each sensor reported at that instant (crashes/)* — i.e. *why* it failed (LiDAR blind, camera saw floor, ...). This is the raw material for improving the visual model and for the robot to reason about its own blind spots.

The repo's name — *meta reasoning* — points at where this goes: a robot that knows the LiDAR misses tables/glass and the camera confuses same-colour walls, **weights its sensors by context**, slows down when its perception is uncertain or degraded, and **transfers** an avoidance learned on one chair to an identical chair elsewhere (analogy). See `AUTONOMOUS_NAVIGATION.md`.

Status

Working: SLAM read, teleop injection, closed-loop motion, reactive A→B, autonomous exploration with coverage bias, collision memory, live cloud+camera viz. Known limits and the improvement roadmap are in `AUTONOMOUS_NAVIGATION.md` (current focus: making the camera/depth layer act *in time*, and moving from reactive wander to frontier exploration with planning).

Disclaimer

Research/interoperability work on the author's own hardware. Not affiliated with Unitree. Moving a bipedal robot autonomously is inherently risky — supervise it, keep a kill switch, use a clear space. # Autonomous Navigation — Algorithm, Lessons & Improvement Roadmap

This document explains how `g1_nav.py` perceives, decides and acts, the hard-won lessons behind each design choice, and a concrete roadmap — ending with the **meta-reasoning** direction that gives the project its name.

1. The control loop (10 Hz)

Every cycle: 1. Read **odometry** (pose `x,y,yaw`) from the captured `slam_mapping/odom`. 2. Query the **LiDAR clean grid** for forward clearance `c0` (and, when escaping, a 360° scan). 3. Read the latest **camera vision** verdict (obstacle ahead? near/far? which side is freer?). 4. Run the **collision detector** (commanded-forward but odom not moving bumped something). 5. Pick a command (`lx, ly, rx, ry`) from a small **behaviour stack** (below). 6. Set `window.__cmd`; an in-page 20 Hz driver publishes it on `rt/wirelesscontroller` with a **600 ms dead-man** (if Python or the bridge dies, the robot stops).

Behaviour priority (highest first): **collision recovery** → **vision avoidance (VAV)** → **coverage redirection** → **reactive obstacle escape (ESC)** → **go forward**.

2. Perception — three complementary layers

No single sensor is enough on a consumer biped; each has a blind spot, so we fuse three.

2.1 LiDAR “clean grid” (the workhorse)

The app's raw decoded cloud is **noisy for navigation**: the robot's own body and the floor (picked up as the head pitches while walking) get baked in as a **phantom ~0.2–0.4 m ring** that follows the robot, which a naïve corridor check reads as “blocked in every direction” → the robot spins like it's drunk. The fix that finally worked is a **purpose-built occupancy grid** built in the page: - **Near-field exclusion**: never map points within ~0.45 m of the robot at capture time (that's where body/pitch-floor noise lives). - **Persistence**: a cell only counts as an obstacle after 2 hits (filters one-off bursts). - **Decay**: every cloud message, all cells lose 1; re-seen cells gain 2 (cap 8). A wall you keep seeing stays; the smear you leave behind **fades in ~3 s**. This single change removed the within-run “drunk” spinning. - Height band chosen to dodge the pitch-floor ([-0.5, +0.8] in the cloud's up axis).

The grid is reset at the start of each **explore** run (the smear otherwise accumulated **across** runs and re-created the ring).

2.2 Camera vision (catches what the LiDAR can't)

LiDAR misses **tables** (horizontal top above the band, thin legs), **glass/whiteboards**, and narrow furniture gaps. The camera (grabbed from the app's WebView `<video>`) adds, in one worker thread: - **Floor segmentation**

(**adaptive**): learns the carpet colour from the strip at the robot’s feet each frame, classifies “floor” by similarity in saturation+brightness, and measures the free-floor fraction per band (left/centre/right). Class-agnostic, lighting-adaptive. - **Edge / thin-obstacle**: a run of consecutive centre columns where the floor is interrupted close a chair/table **leg** the floor-fraction average would miss. - **YOLOv8s**: furniture classes (table, chair, sofa, fridge...) — fires on smaller/off-centre boxes because *any* furniture is an obstacle; box position/size gives a near/mid/far estimate. - **MiDaS monocular depth**: the robust, class-agnostic layer — compares depth at mid-image-height vs the floor at the feet; if the “floor” at mid-height is as close as the near floor, there’s a **vertical surface ahead** (wall/whiteboard/glass) regardless of colour. Runs on Apple-GPU (MPS).

2.3 Collision detector (the safety net / teacher)

When nothing geometric saw it, odometry does: **commanded forward for 1.5 s but moved <5 cm** collision. The robot then backs up (rear-checked), turns to the freer side, nudges forward to leave the spot, **and injects the obstacle into the grid** so it won’t repeat — plus saves a labelled crash snapshot.

3. Control — moving a biped you don’t “own”

You can’t command joints on the Air (low-level is EDU-only and not actuated). You command **gait velocity**: `rt/wirelesscontroller {lx,ly,rx,ry}` at 20 Hz, injected on the app’s datachannel. Key facts learned the hard way: - **Dead-zone**: the robot ignores |value| below ~0.3; the app’s joystick never sends below ~0.5. Use ~0.4. - **Format**: `{"type":"msg","topic":"rt/wirelesscontroller","data":{"lx":...,"ly":...,"rx":...,"ry":...}}` on the datachannel labelled `data`. - **Axes**: `+ly` forward, `+rx` turns right (yaw decreases), `+lx` strafes. Robot is **slow** (~0.12 m/s) — which is good for reaction margin. - The robot’s **own balance policy** keeps it upright; we only set desired velocity.

4. Exploration

Reactive wander with two coverage mechanisms: - **Novelty bias**: track visited 0.4 m cells; when blocked, among *open* directions pick the one whose ray (sampled to 3 m) is **least-visited**. - **Periodic redirection**: even with a clear front, every ~6 s, if a clearly more-open and less-visited heading exists, commit a turn toward it. - Visited coverage is **persisted in the page** (`window.__visited`) so it survives Python restarts within the same SLAM session.

5. Lessons (what actually moved the needle)

1. **Don’t trust the raw cloud for control**. Build your own grid with near-field exclusion + persistence + **decay**. Decay was the difference between “drunk” and smooth.
2. **Sensor fusion beats any single sensor**, but only if each is *cheap and timely*. A great detector that arrives stale is worse than none (see §6).
3. **Collisions are data, not just failures**. Inject-on-bump + a labelled crash dataset turns every mistake into map memory and training signal.
4. **Reactive complete**. Wander + novelty covers more but still gets wedged; systematic coverage needs a map and a planner (§6).
5. **Calibrate from real logs**. Each detector got tuned from `vsee` + crash `.txt` (e.g. the carpet is near-grey S 5; white walls are the same colour → only depth separates them).

6. Improvement roadmap

Near-term (close the current gaps)

- **Make vision act in time**. Right now MiDaS+YOLO can run but the verdict arrives too late to avoid (it only confirms the bump). Fixes: run vision on MPS at low res; **react proportionally at medium**

distance (slow + steer) instead of a binary block-when-close; widen the freshness window for a slow robot; and **lower forward speed when the vision pipeline is degraded** (self-monitoring, see meta-reasoning). The VHEALTH log line already measures per-frame time and staleness.

- **Unified costmap.** Fuse LiDAR-grid + camera-projected obstacles + depth into **one** world-frame costmap with confidence and decay; the controller/planner reads only that. Class-agnostic by design (“there is something at distance d in direction ”).
- **Metric depth.** MiDaS is relative; use the known floor plane (it must lie below the horizon and recede) to fit scale $\rightarrow$ metric obstacle distance, so thresholds are physical, not tuned per scene.

Mid-term (from reactive to deliberative)

- **Frontier exploration + planning.** Maintain free/occupied/unknown; detect frontiers (free unknown boundary); plan a path (A/D-lite on the costmap) to the nearest frontier. Guarantees coverage and **avoids dead-ends/narrow gaps** the reactive wander walks into.
- **Recovery behaviours library** with explicit pre/post-conditions instead of fixed timers.

Long-term — meta-reasoning (the project’s namesake)

The robot reasoning about *its own cognition*, which is exactly where a consumer robot with unreliable, partial perception needs to live:

1. **Sensor reliability model (context-conditioned).** The robot *knows* the LiDAR misses tables/glass and the camera confuses same-colour walls. Learn, per context (open room vs furniture cluster vs kitchen), **which sensor to trust** and weight the fused costmap accordingly.
2. **Confidence-aware action (metacognitive control).** Act on *perceptual confidence*: when sensors disagree or the scene matches a known-hard context, **slow down / widen margins**; when confidently clear, move fast. Uncertainty $\rightarrow$ caution is the metacognitive loop.
3. **Self-monitoring of the perception process.** Track latency/staleness/sensor-liveness (the VHEALTH/odom-frozen guards are seeds of this) and adapt behaviour when degraded — the robot noticing “*my eyes are lagging, I should slow down.*”
4. **Learning from failure + analogy (the PhD link).** We literally saw the robot avoid one piece of furniture and then hit an **identical** one elsewhere. A meta-reasoner recognises the *analogy* (“this is like the chair I avoided”) and **transfers** the avoidance. Concretely: embed crash snapshots, match new views by similarity, and pre-emptively avoid analogous obstacles — and/or fine-tune the detector on the actual furniture in this environment from the crash dataset.
5. **Explanation.** Because every decision is fused from named signals (LiDAR grid, depth_ratio, YOLO label, floor fraction), the robot can **explain** *why* it turned or *why* it failed — a substrate for introspective reasoning and for debugging perception.

The throughline: on hardware that gives you one noisy channel and three blind-spotted sensors, the winning move isn’t a better single sensor — it’s a system that **knows what it doesn’t know** and acts accordingly.